

Programação Avançada

Perguntas das aulas teóricas

Pergunta 2025.02.10

- Qual ou quais das seguintes ferramentas permitem executar um programa em *Java*?
- ✓ **A** – `java`
- ✗ **B** – `javac`
- ✗ **C** – `javap`
- ✗ **D** – `javarun`
- ✗ **E** – Não é necessária qualquer ferramenta adicional, porque o *Java* é *cross-platform*

Pergunta 2025.02.14

- Qual ou quais das seguintes instruções permitem criar *arrays* em *Java*?

✗ **A** – `int [] array1;`

✗ **B** – `int array2[];`

✓ **C** – `int [] array3 = new int[4];`

✓ **D** – `int array4[] = {2, 4, 6, 8};`

✗ **E** – `int array5[4];`

Pergunta 2025.02.17

- Qual ou quais das seguintes condições serão sempre falsas, sabendo que os valores das variáveis *i* e *d* são obtidos da seguinte forma:

```
int i = (int) Math.random() * 100 + 10;  
double d = Math.random() * 10 + 5;
```

✓ **A** – *i* == 90 && *d* == 5.00

✓ **B** – *i* == 10 && *d* == 15.00

✗ **C** – *i* == 10 && *d* == 7.89

✓ **D** – *i* == 20 && *d* == 10.98

✓ **E** – *i* == 120 && *d* == 20.00

Pergunta 2025.02.21

- Qual ou quais das seguintes hipóteses podem ser *construtores* de instâncias da classe `Pessoa`

✗ **A** – `init(String nome, int idade) { ... }`

✓ **B** – `Pessoa() { ... }`

✗ **C** – `create(String nome, int idade) { ... }`

✗ **D** – `static { ... }`

✓ **E** – `Pessoa(String nome, int idade) { ... }`

Pergunta 2025.02.24

- Sabendo que foi definida uma classe `Pessoa` no *package* `pt.isec.pa.modelo`, qual ou quais das seguintes instruções podem ser usadas para importar a classe referida no contexto de um outro *package*

✗ **A** – `include pt.isec.pa.modelo;`

✓ **B** – `import pt.isec.pa.modelo.*;`

✗ **C** – `import pt.isec.pa.modelo;`

✗ **D** – `include pt.isec.pa.modelo.Pessoa;`

✗ **E** – `import pt.isec.pa.modelo.P*;`

Pergunta 2025.03.03

- Qual ou quais das seguintes afirmações estão corretas?
 - ✓ **A** – Em *Java* não é permitida herança múltipla de classes
 - ✗ **B** – Em *Java* apenas é permitida herança múltipla de classes quando uma das classes base é a `Object`
 - ✗ **C** – Em *Java* apenas é permitida herança múltipla de, no máximo, 2 classes, sendo obrigatoriamente uma delas a classe `Object`
 - ✓ **D** – Em *Java* todas as classes derivam, direta ou indiretamente, da classe `Object`
 - ✗ **E** – Todas as afirmações anteriores estão incorretas

Pergunta 2025.03.07

- Assumindo a existência da seguinte classe:

```
class Pessoa {  
    String nome;  
    int idade;  
    ...  
    @Override  
    boolean equals(Object o) {  
        if (this == o) return true;  
        if (o == null || getClass() != o.getClass()) return false;  
        Pessoa other = (Pessoa) o;  
        return nome.equalsIgnoreCase(other.nome) && Math.abs(idade-other.idade) < 5;  
    }  
}
```

- Qual ou quais das seguintes instruções seriam válidas para a implementação da função hashCode dessa classe?

- X A** - return nome.hashCode();
- X B** - return nome.hashCode()+idade;
- ✓ C** - return nome.toUpperCase().hashCode();
- X D** - return nome.toUpperCase().hashCode()+idade;
- ✓ E** - return 123;

Pergunta 2025.03.10

- Assumindo a existência das seguintes classes e interfaces:

```
class ClasseA {...}  
class ClasseB {...}  
interface InterfaceA {...}  
interface InterfaceB {...}
```

- Qual ou quais das seguintes declarações da classe ClasseC são válidas?

- ✓ **A** - `class ClasseC extends ClasseA {...}`
- ✗ **B** - `class ClasseC extends ClasseA, ClasseB {...}`
- ✓ **C** - `class ClasseC extends ClasseA implements InterfaceA {...}`
- ✓ **D** - `class ClasseC extends ClasseA implements InterfaceA, InterfaceB {...}`
- ✗ **E** - `class ClasseC extends ClasseA, ClasseB implements InterfaceA {...}`

Pergunta 2025.03.14

1. Quais são os principais propósitos do *Git*?

- ☐ **A** – Gerir bases de dados
- ☒ **B** – Controlo de versões
- ☐ **C** – Executar servidores
- ☒ **D** – Permitir que vários programadores colaborem num projeto
- ☐ **E** – Criar interfaces de utilizador

Pergunta 2025.03.14

2. Quais das seguintes afirmações são verdadeiras?

- ✗ **A** – O comando *git commit* envia alterações para o repositório remoto
- ✓ **B** – O comando *git push* envia *commits* locais para o repositório remoto
- ✓ **C** – O comando *git pull* traz alterações do repositório remoto para o local
- ✓ **D** – Os conflitos nas operações *merge* ocorrem quando as mesmas linhas são alteradas em ambas as *branches*
- ✗ **E** – Uma operação *merge* é concluída automaticamente sem necessidade de intervenção em caso de conflito

Pergunta 2025.03.17

- Tendo em consideração a seguinte declaração de classes, quais das afirmações são verdadeiras?

```
class ABC {  
    class EFG { }  
    static class HIJ { }  
}
```

- ✓ **A** – A classe `EFG` não pode ser instanciada fora do contexto de uma instância de `ABC`
- ✗ **B** – A classe `EFG` pode ser instanciada sem a necessidade de uma instância de `ABC`
- ✗ **C** – A classe `EFG` é uma *nested class* e a classe `HIJ` é uma *inner class*
- ✓ **D** – A classe `EFG` é uma *inner class* e a classe `HIJ` é uma *nested class*
- ✗ **E** – A classe `HIJ` pode aceder métodos de instância da classe `ABC`, mas não pode aceder aos métodos estáticos de `ABC`

Pergunta 2025.03.21

- Qual será o *output* deste programa?

X A – A12367Z

X B – A124Z

⚠ C – A1246Z

X D – A1257Z

✓ E – A12567Z

```
public class Main {  
    public static void func() {  
        Integer n=null;  
        System.out.print("1");  
        try {  
            System.out.print("2");  
            int i = (int) (1234 / n);  
            System.out.print("3");  
        } catch (ArithmeticException e) {  
            System.out.print("4");  
            return;  
        } catch (NullPointerException e) {  
            System.out.print("5");  
        } finally {  
            System.out.print("6");  
        }  
        System.out.print("7");  
    }  
    public static void main(String[] args) {  
        System.out.print("A");  
        func();  
        System.out.println("Z");  
    }  
}
```

Pergunta 2025.03.24

- Qual das seguintes operações é essencial realizar depois de abrir um ficheiro em *Java*?
 - ✗ **A** – Verificar se o ficheiro foi aberto corretamente
 - ✗ **B** – Ler a informação do ficheiro
 - ✗ **C** – Escrever informação no ficheiro
 - ✓ **D** – Fechar o ficheiro
 - ✗ **E** – Apagar o ficheiro

Pergunta 2025.03.28

- Qual das seguintes opções permite configurar uma classe `Xpto` como sendo serializável?

X A – `class Xpto extends Serializable {...}`

X B – `class Xpto {
 static final long serialVersionUID = 100L;
 ...
}`

✓ C – `class Xpto implements Serializable {...}`

X D – Qualquer uma das anteriores

X E – Nenhuma das anteriores

Pergunta 2025.03.31

- No âmbito do ciclo de vida de um objecto `Application` do *JavaFX*, qual é o primeiro método que é chamado após a criação do objecto e execução do construtor da classe?

X A – `begin`

✓ B – `init`

X C – `run`

X D – `start`

X E – `stop`

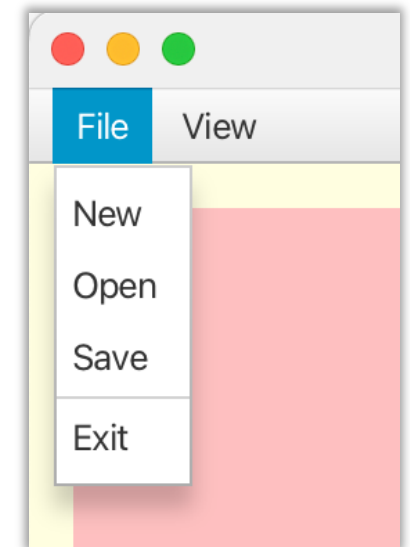
Pergunta 2025.04.07

- Quais são os elementos necessários para implementar um componente da UI em *JavaFX*, segundo o modelo *MVC@PA*?
 - ✓ **A** – Construtor que chama os métodos `createViews()`, `registerHandlers()` e `update()`
 - ✓ **B** – Método `createViews()`
 - ✓ **C** – Método `registerHandlers()`
 - ✓ **D** – Método `update()`
 - ✗ **E** – Nenhum dos anteriores

Pergunta 2025.04.11

- A opção "*File*" do menu na figura ao lado pode ser criada com que instrução?

- ☒ **A** – `new MenuBar("File")`
- ☒ **B** – `new Menu("File")`
- ☒ **C** – `new MenuItem("File")`
- ☒ **D** – Qualquer uma das opções anteriores
- ☒ **E** – Nenhuma das opções anteriores



Pergunta 2025.05.02

- Assumindo a existência de um objeto `PropertyChangeSupport` devidamente criado e com um conjunto de `PropertyChangeListeners` registados, qual ou quais das seguintes instruções sinalizarão os *Observers/Subscribers* da existência de alterações no *Observable/Publisher*?
 - ✓ **A** – `firePropertyChange (null, 1, 2)`
 - ✓ **B** – `firePropertyChange ("DEIS", "DEIS", "deis")`
 - ✓ **C** – `firePropertyChange ("width", 2.0000, 2.0001)`
 - ✗ **D** – `firePropertyChange (null, 2, 2)`
 - ✓ **E** – `firePropertyChange (null, null, null)`

Pergunta 2025.05.05

- Quantas instâncias da classe `Stage` podem existir numa aplicação *JavaFX*?
 - ✗ **A** – Uma, a instância que é recebida como parâmetro no método `start`
 - ✗ **B** – Uma, a instância que é recebida como parâmetro no método `start` ou uma que seja criada no contexto da aplicação
 - ✗ **C** – Duas, a instância que é recebida como parâmetro no método `start` e, adicionalmente, uma que seja criada no contexto da aplicação
 - ✗ **D** – Qualquer número de instâncias, desde que sejam todas criadas dentro do método `start`
 - ✓ **E** – Qualquer número de instâncias, sendo apenas limitado pela quantidade de recursos disponíveis

Pergunta 2025.05.09

- Quando se pretendem gerar sons numa aplicação *JavaFX*, qual o módulo que é necessário indicar no parâmetro `--add-modules` aquando da execução dessa aplicação?

X A – `javafx.fxml`

X B – `javafx.graphics`

✓ C – `javafx.media`

X D – `javafx.sounds`

X E – `javafx.swing`

Pergunta 2025.05.12

1. Sobre o padrão *Command*, assinale as afirmações corretas:

- ✓ **A** – Permite encapsular uma solicitação como um objeto, separando o emissor do recetor da ação
- ✓ **B** – É utilizado para implementar funcionalidades como desfazer/refazer
- ✗ **C** – Obriga a que todas as operações tenham um único recetor
- ✓ **D** – Facilita o registo e execução de comandos em momentos distintos
- ✗ **E** – Requer herança múltipla para funcionar corretamente

2. Sobre o padrão *Memento*, assinale as afirmações corretas:

- ✓ **A** – Permite guardar o estado interno de um objeto sem violar o encapsulamento
- ✗ **B** – Exige acesso público a todos os atributos do *originador*
- ✓ **C** – É frequentemente utilizado em funcionalidades de desfazer/retroceder
- ✗ **D** – O *caretaker* conhece a estrutura interna do *memento*
- ✓ **E** – O *originador* é responsável por criar e restaurar os *mementos*

Pergunta 2025.05.16

- Qual é a *framework* de testes a usar na última etapa do trabalho prático?

✗ **A** – *PATestUnit*

✗ **B** – *PAUnit*

✗ **C** – *PATest*

✗ **D** – *TestPA*

✓ **E** – *JUnit*

Pergunta 2025.05.19

- Qual ou quais das seguintes afirmações sobre o uso de *Javadoc* estão corretas?
 - ✓ **A** – O *Javadoc* permite gerar documentação HTML a partir de comentários estruturados no código fonte *Java*
 - ✗ **B** – A etiqueta `@param` é usada para documentar o valor de retorno de um método
 - ✓ **C** – Comentários *Javadoc* devem começar com `/**` e terminar com `*/`
 - ✗ **D** – A etiqueta `@author` é obrigatória em todas as classes
 - ✗ **E** – Comentários simples iniciados com `//` também são incluídos automaticamente na documentação gerada pelo *Javadoc*

Pergunta 2025.05.23

- Que modificador de acesso deve ser associado ao método `changeState` da classe `Context` do padrão *State* (*Finite State Machine*)?

☐ **A** – `private`

☒ **B** – `package-private`

☐ **C** – `protected`

☐ **D** – `public`

☐ **E** – Qualquer uma das anteriores